

GOVERNMENT COLLEGE OF ENGINEERING



TIRUNELVELI – 627 007.

20 - 20

Register No.

CERTIFICATE

This is a Bonafide record of work done by

**..... Government College of Engineering,
Tirunelveli during the year 20- 20**

STATION: TIRUNELVELI – 7.

DATE:

Staff-in-Charge

Head of the Department

Submitted for the Anna University Practical Examination held at

Government College of Engineering, Tirunelveli on

.....

Internal Examiner

External Examiner

TABLE OF CONTENTS

S.No	Date	Title	Page No.	Marks	Signature
1.		TESTING LAB SETUP	1		
2.		GIT OPERATIONS	3		
3.		CREATING PROJECT IN SONAR QUBE	6		
4.		USING SONARQUBE WITH SONAR-RUNNER	8		
5.		CREATING A LOCAL REPOSITORY IN ARTIFACTORY	10		
6.		BUILD AUTOMATION USING MAVEN	12		
7.		JENKINS INSTALLATION AND SYSTEM CONFIGURATION	16		
8.		DOWNLOAD THE PLUGINS IN JENKINS	18		
9.		CREATING CENTRAL CI - PIPELINE	20		
10.		COPYING AND MOVING JOBS	22		
11.		CREATING PIPELINE VIEW IN JENKINS	24		
12.		CONFIGURING GATE CONDITIONS	26		

EX.NO: 1

DATE:

TESTING LAB SETUP

AIM

To successfully configure the system environment variables and test the operation of the required DevOps tools by verifying their user interfaces.

PROCEDURE

1. Configuring System Environment Variables

- Open the System Environment Variables by right-clicking on "This PC", selecting "Properties", then "Advanced System Settings", and clicking on "Environment Variables...".
- Add the following System Variables if they are not already present:
 - JAVA_HOME = path to the JDK folder (e.g., C:\Program Files\Java).
 - M2_HOME = path to the Maven folder (e.g., C:\Program Files\Maven).
- Edit the existing PATH variable to include: %JAVA_HOME%/bin; %M2_HOME%/bin; <path to sonar-runner>\sonar-runner-2.4\bin;.

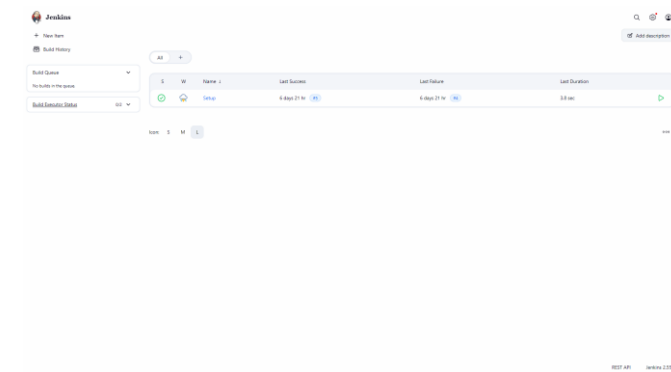
2. Starting the Services

- Start all the installed tools by executing their respective batch files.

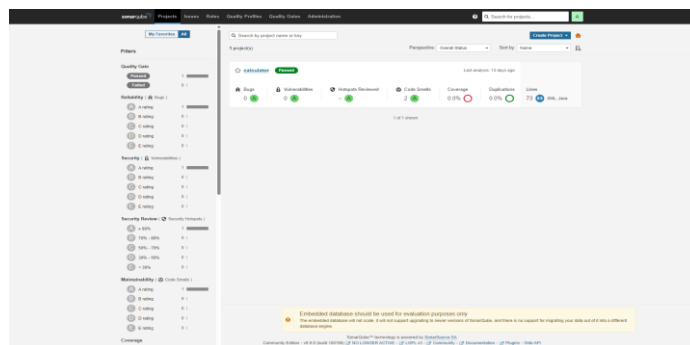
3. Verifying Tool Interfaces

- Ensure all applications are working by accessing their user interfaces via a web browser with the following URLs and credentials:
 - **SonarQube:** http://localhost:9000 [Credentials: admin/admin].
 - **Tomcat:** http://localhost:8080 [Credentials: tomcat/s3cret].
 - **Jenkins:** http://localhost:8064/jenkins.

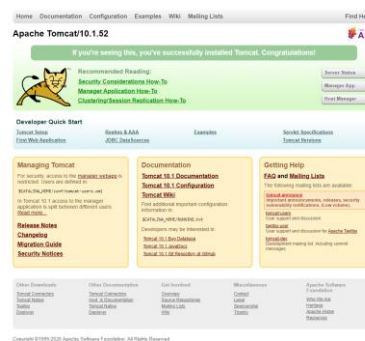
OUTPUT



Jenkins: <http://localhost:8064/jenkins/>



SonarQube: <http://localhost:9000/dashboard>



Apache Tomcat Server: <http://localhost:8080/>

RESULT

The testing lab environment was successfully set up and the tools were tested and verified for the forthcoming Jenkins exercises.

EX.NO: 2

DATE:

GIT OPERATIONS

AIM

To perform the basic operations on Git repositories using EGit (the Eclipse plugin).

PROCEDURE

1) Pulling code from Git repository

- **Step 1:** Go to Eclipse -> File -> Import -> Git -> Projects from Git.
- **Step 2:** Select "Clone URI" (or central repository details) and enter your repository URL along with your credentials (Username and Password).
- **Step 3:** Click Next and select the specific branch you want to clone from the remote repository.
- **Step 4:** Click Next and browse for a local directory destination to store the local repository.
- **Step 5:** Click Next, choose the wizard "Import existing Eclipse projects" (the first option), and click Next again.
- **Step 6:** Click Finish. You will now be able to observe the cloned project in your Project Explorer view (e.g., JNTU_Calc_Application or cicc-calculator).

2) Commit Changes from Eclipse to Local Git Repository

- **Step 1:** Expand your imported project in the Project Explorer (e.g., navigate to `src\main\webapp\index.jsp`), make some changes to the code, and save the file. You will observe a ">" symbol appear on the project and the edited source file indicating uncommitted changes.
- **Step 2:** Right-click on the project -> Team -> Commit. Drag the modified files from the "Unstaged Changes" pane to the "Staged Changes" pane.
- **Step 3:** Enter an appropriate commit message in the provided text box and click on the "Commit" button. The ">" symbol will disappear, confirming the commit. You can commit multiple individual changes to the local repository using this method.

3) Push Changes from Local Git Repository to Remote Git Repository

- **Step 1:** Right-click on the project -> Team -> click on "Push to upstream".
- **Step 2:** A push results window will appear confirming that the changes were successfully pushed from the local repository to the central repository.
- *Note:* If the push results in a "non fast-forward" rejection (meaning other developers have pushed changes to the central repository in the meantime), you must perform a Fetch from upstream, merge with the local branch, resolve any conflicts, commit, and then push again. You can view the revision history by navigating to Project -> Team -> "Show In History".

4) Pull Changes from Remote Git Repository to Local Git Repository

- *Assumption:* A team member has committed changes to the remote repository.
- **Step 1:** Right-click on the project -> Team -> Pull. Enter your credentials if prompted.
- **Step 2:** You can observe the fetch result and merge input showing the updates pulled down.
- *Note:* The "Pull" command actually performs two actions in sequence: Fetch and Merge. If there are any merge conflicts after the pull operation, you must resolve them and commit again. You can observe the updated revision history using Project -> Team -> "Show in History".

5) Creating branches on local repository from Eclipse via EGit plugin

- **Step 1:** Right-click on the Project in the Project Explorer and go to Team -> Switch to -> New Branch....
- **Step 2:** The parent branch will by default be the branch that is currently open in Eclipse. You can change it by clicking on the "Select" option.
- **Step 3:** Name the New Branch and ensure the "Check out new branch" box is checked if you want to switch to the new branch immediately.
- **Step 4:** Click on Finish.

OUTPUT

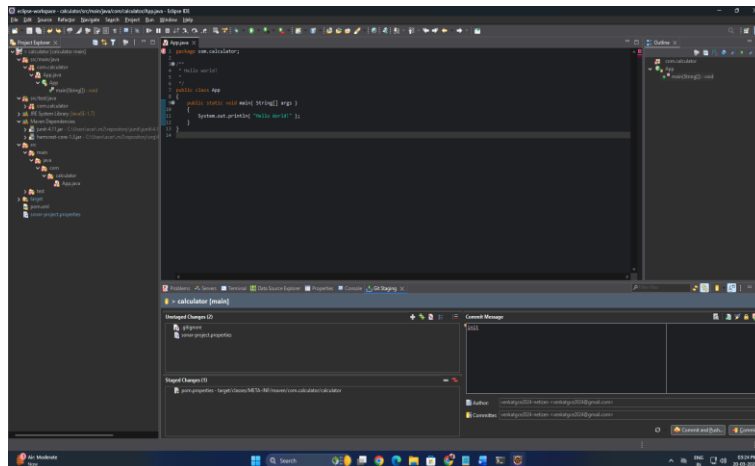


Fig 2.1 Initializing Git

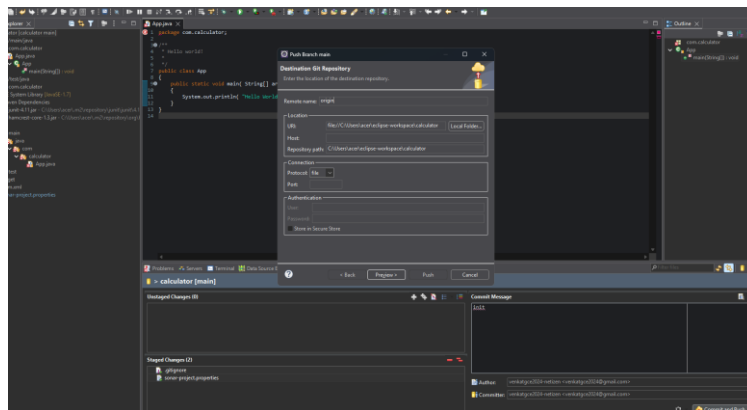


Fig 2.2 Creating Branch

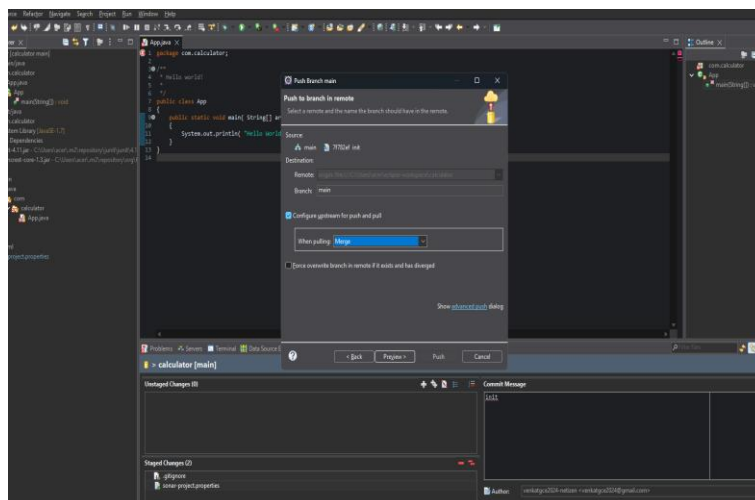


Fig 2.3 Pushing Changes

RESULT

The basic operations with Git related to version control, including pulling, committing, pushing, and branching, were successfully performed and verified.

EX.NO: 3

DATE:

CREATING PROJECT IN SONAR QUBE

AIM

To understand the process of creating a new project within the SonarQube dashboard and generating an authentication token to link it with a local source code repository.

PROCEDURE

Logging In and Initializing the Project

- **Step 1:** Open the SonarQube web interface (usually <http://localhost:9000>) and log in using the default credentials (admin / admin or admin / password).
- **Step 2:** On the SonarQube dashboard, click on the **Manually** option to create a new project from scratch.
- **Step 3:** Enter the project display name and project key (e.g., calculator) in the respective fields, and then click **Set Up**.

Generating the Authentication Token

- **Step 4:** When asked how you want to analyze your repository, click on the **Locally** option.
- **Step 5:** In the "Provide a token" section, click on the **Generate** button to create a unique security token for this project.
- **Step 6:** The generated token will be displayed on the screen. **Copy this token** and save it somewhere secure on your system, as it cannot be retrieved later. Click **Continue**.

Configuring the Build Tool

- **Step 7:** Under the "Run analysis on your project" section, select **Maven** as the option that best describes your build.
- **Step 8:** Open your project's pom.xml file in your IDE and paste the copied token within the `<sonar.login>` tag (e.g., `<sonar.login>Paste the token here</sonar.login>`) to securely link your build to this SonarQube project.

OUTPUT

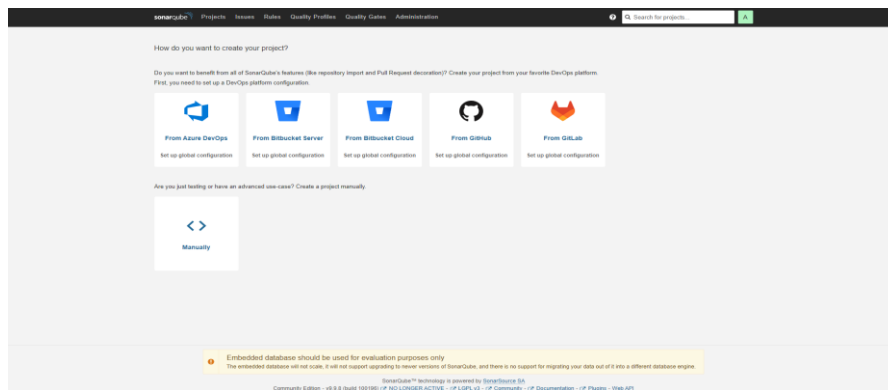


Fig 3.1 SonarQube

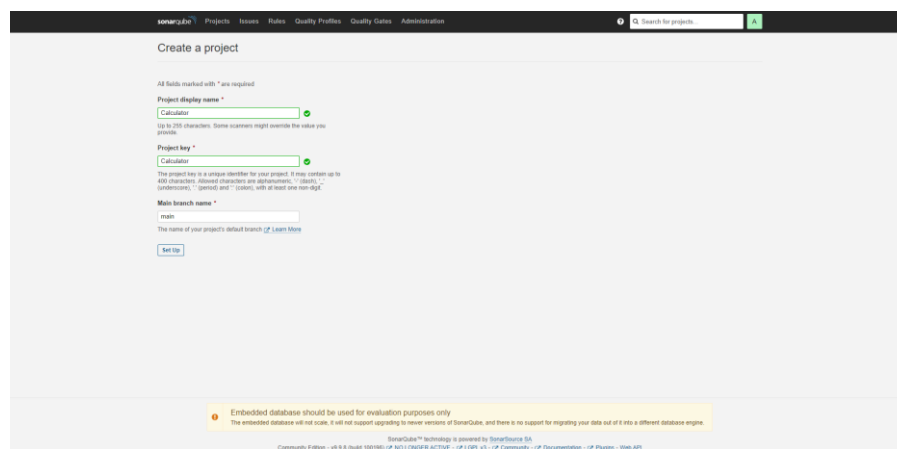


Fig 3.2 Creating Project

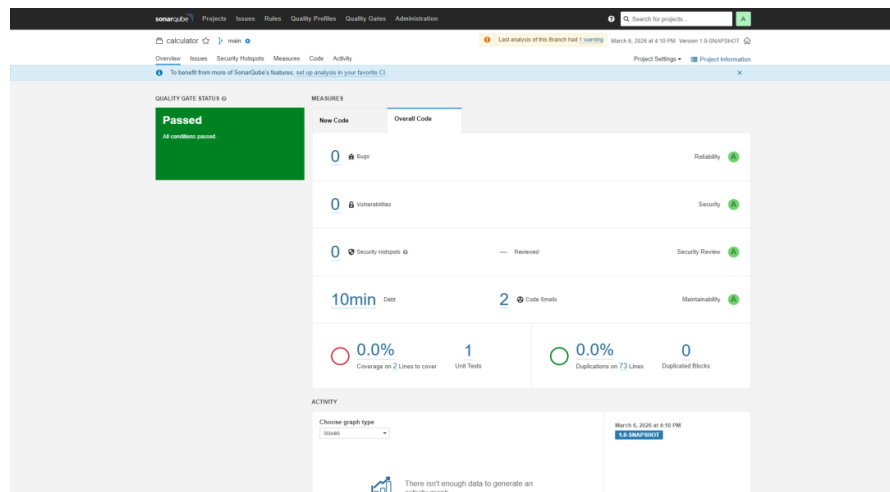


Fig 3.3 Project Analysis

RESULT

A new project was successfully created and configured within the SonarQube server, and its authentication token was generated and mapped to the project's POM file.

EX.NO: 4

DATE:

USING SONARQUBE WITH SONAR-RUNNER

AIM

To understand how to run SonarQube static code analysis using the Sonar-Runner command-line tool.

PROCEDURE

Starting the Server and Logging In

- **Step 1:** Start the SonarQube server by executing the appropriate batch file (StartSonar.bat).
- **Step 2:** Once the server has started successfully, open a web browser, navigate to the SonarQube dashboard at <http://localhost:9000>, and log in using the credentials admin / admin.

Configuring Sonar-Runner Properties

- **Step 3:** Navigate to the conf folder within your Sonar-Runner installation directory.
- **Step 4:** Edit the sonar-runner.properties file to include your specific project details and database connections. You will need to configure properties such as :
 - sonar.host.url=<http://localhost:9000>
 - sonar.sourceEncoding=UTF-8
 - sonar.login=admin
 - sonar.password=admin
 - sonar.projectKey=Your_project_key
 - sonar.projectName=Your_project_name
 - sonar.projectVersion=Your_project_version
 - Source locations and binary locations according to your project's path.

Executing the Analysis

- **Step 5:** Open a command prompt directly inside your project's source directory (e.g., src folder).
- **Step 6:** Run the sonar-runner command in the terminal to initiate the static code analysis. Wait for the execution to complete successfully.

Observing and Resolving Issues

- **Step 7:** Return to the SonarQube dashboard in your browser. Select your project name from the list of analyzed projects to view the latest report.
- **Step 8:** Observe the static code analysis report, noting any critical issues, bugs, vulnerabilities, or code smells.
- **Step 9:** Resolve the identified technical issues in your source code, rerun the Maven build, and observe the changes to the technical debt value on the dashboard.

OUTPUT

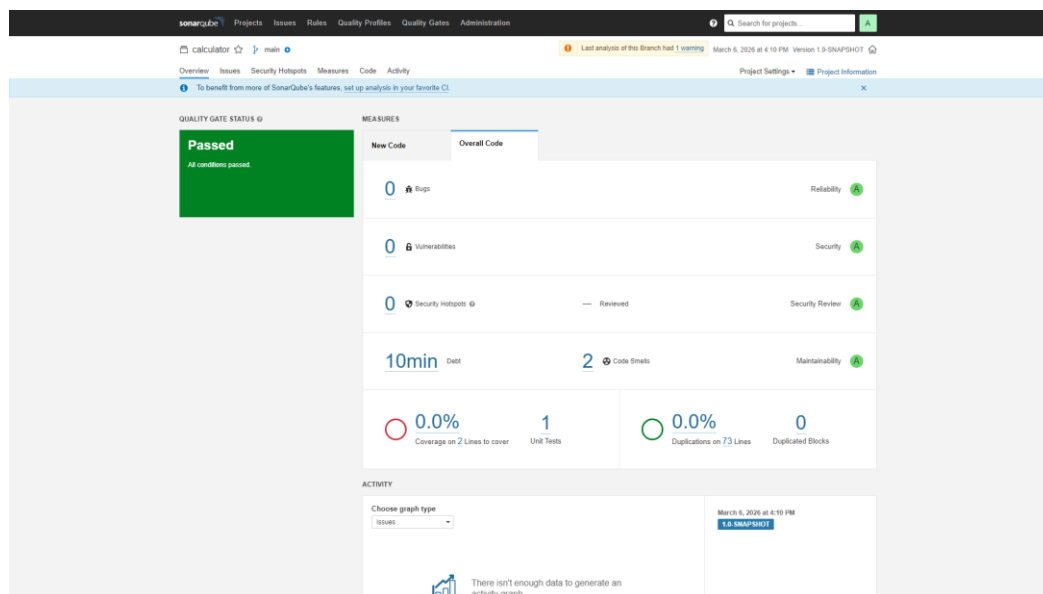


Fig 4.1 Analyzed Project

RESULT

The static code analysis was successfully executed using the Sonar-Runner command-line tool, and the results were observed and analyzed on the SonarQube dashboard.

EX.NO: 5

DATE:

CREATING A LOCAL REPOSITORY IN ARTIFACTORY

AIM

To understand the creation of a local repository in Artifactory for storing build packages.

PROCEDURE

Logging In and Navigating

- **Step 1:** Go to the Artifactory URL and log in using the credentials admin / Password1!.
- **Step 2:** Navigate to Administration -> Repositories -> Repositories -> Add repositories -> Local Repository.

Configuring the Local Repository

- **Step 3:** Select the package type as **Maven**.
- **Step 4:** To add the repository key, check your pom.xml file and copy the <name> tag value (e.g., Calc_Dev_Snapshot) to use as the repository key.
- **Step 5:** Click on **Create Local Repository** to finalize the setup.

Verifying the Repository

- **Step 6:** You can view the stored binaries and the newly created repository in Artifactory under Application -> Artifactory -> Packages.

OUTPUT

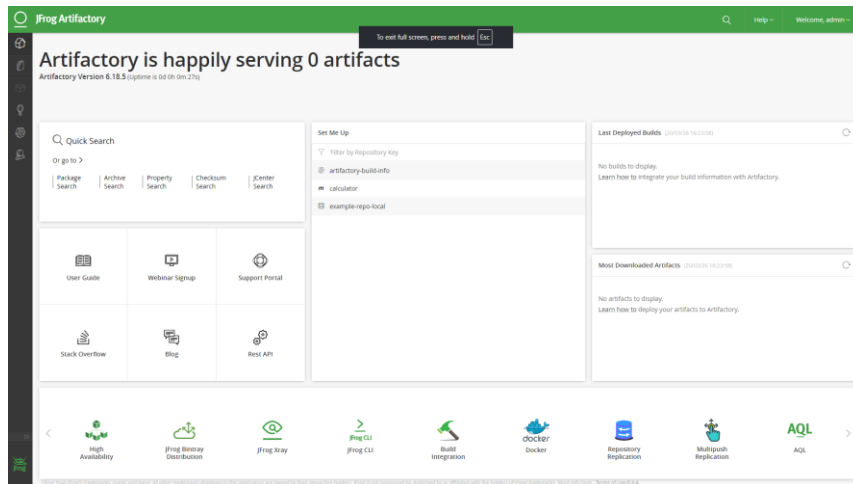


Fig 5.1 Artifactory Dashboard

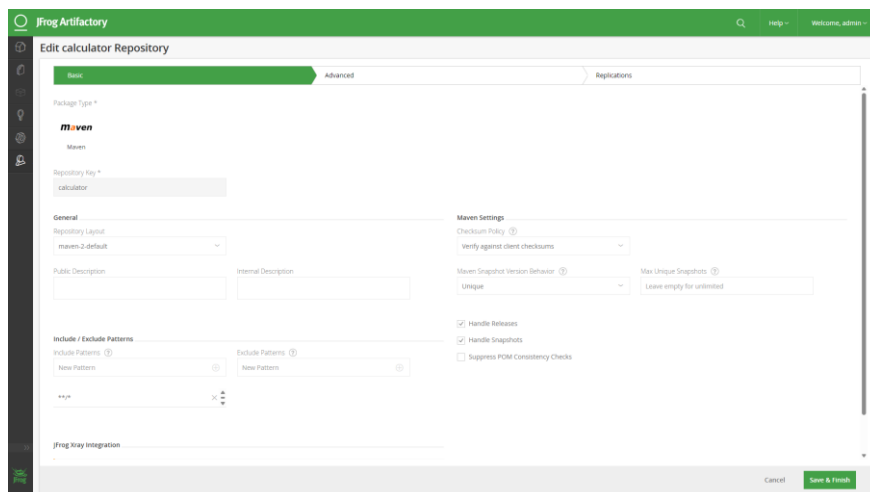


Fig 5.2 Creating Repository

RESULT

A local Maven repository was successfully created in Artifactory to securely store application binaries and packages.

EX.NO: 6

DATE:

BUILD AUTOMATION USING MAVEN

AIM

To understand build automation by writing a script in Maven with specific goals to invoke activities in a Continuous Integration (CI) pipeline.

PROCEDURE

Configuring the Maven POM File

- **Step 1:** Open your project's pom.xml file.
- **Step 2:** Ensure the basic project information (groupId, artifactId, packaging as war, version) is configured correctly.
- **Step 3:** Add the required dependencies for compiling and testing the application. This includes:
 - junit for unit testing.
 - javax.servlet-api for web servlets.
 - selenium-java for automated UI testing.
- **Step 4:** Configure the <build> section with the necessary plugins to automate the pipeline stages:
 - maven-war-plugin to package the application into a deployable web archive.
 - sonar-maven-plugin to link the build to SonarQube for static code analysis.
 - jacoco-maven-plugin to generate code coverage reports during the testing phase and enforce coverage limits (e.g., setting a minimum COVEREDRATIO of 0.10).
- **Step 5:** Set up specific <profiles> to handle different testing phases such as Unit Testing (ut), Integration Testing (it), and Performance Testing (pt using jmeter-maven-plugin).

Executing Maven Goals

- **Step 6:** Open a command prompt or terminal in the project's root directory (where the pom.xml is located).
- **Step 7:** Execute the comprehensive build command to trigger the entire automated sequence:

Bash

```
mvn clean compile test jacoco:report sonar:sonar war:war
```

- **Step 8:** Observe the console output. It will download required packages, compile the code, run the JUnit tests, generate JaCoCo reports, push the analysis to SonarQube, and finally package the .war file. Look for the [INFO] BUILD SUCCESS message at the end.

Observing the Build Artifacts

- **Step 9:** Navigate to the newly generated target folder inside your project directory to verify the results of the Maven goals:
 - **JUnit test reports:** Located in the surefire-reports folder (contains .xml and .txt files).
 - **JaCoCo code coverage reports:** Located under site/jacoco/ (open index.html to view the visual report).
 - **Application Package:** The final deployable archive (e.g., calculator.war) is generated directly in the target folder.

OUTPUT

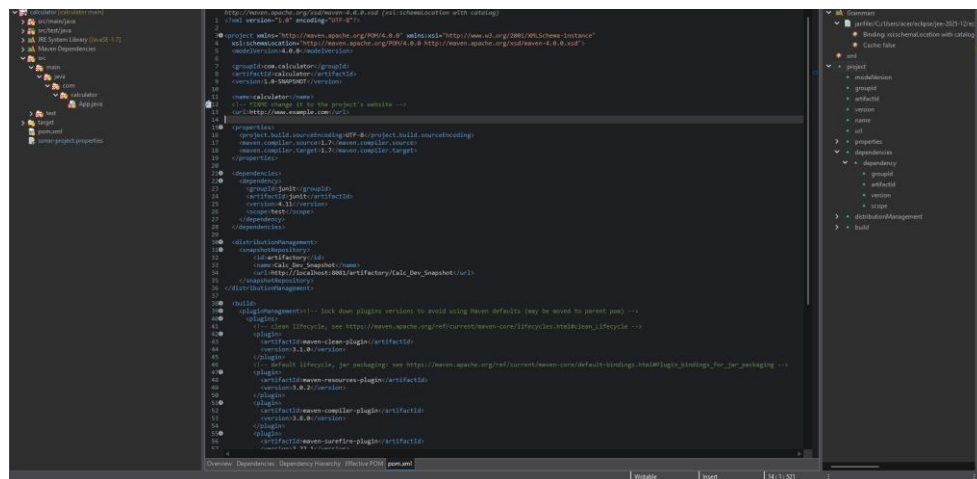


Fig 6.1 pom.xml

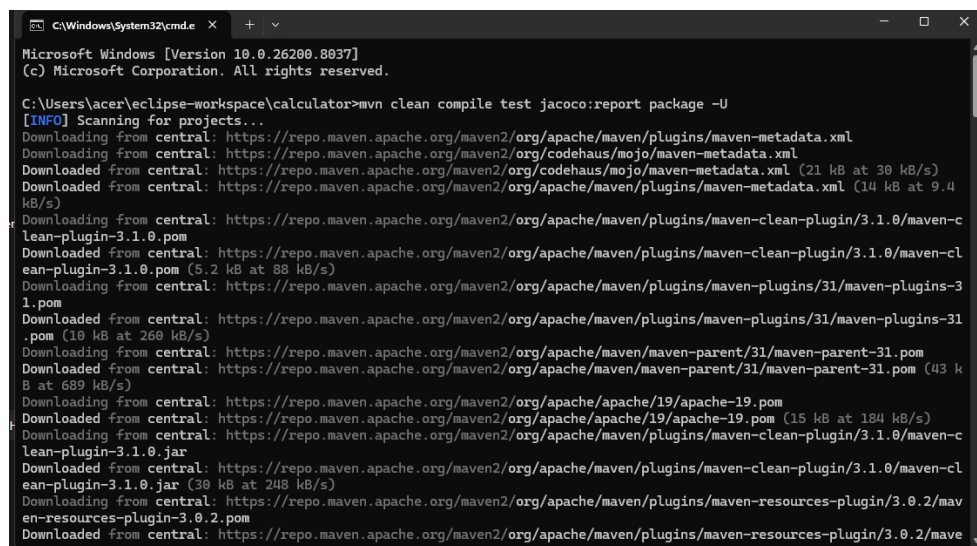


Fig 6.2 Compiling

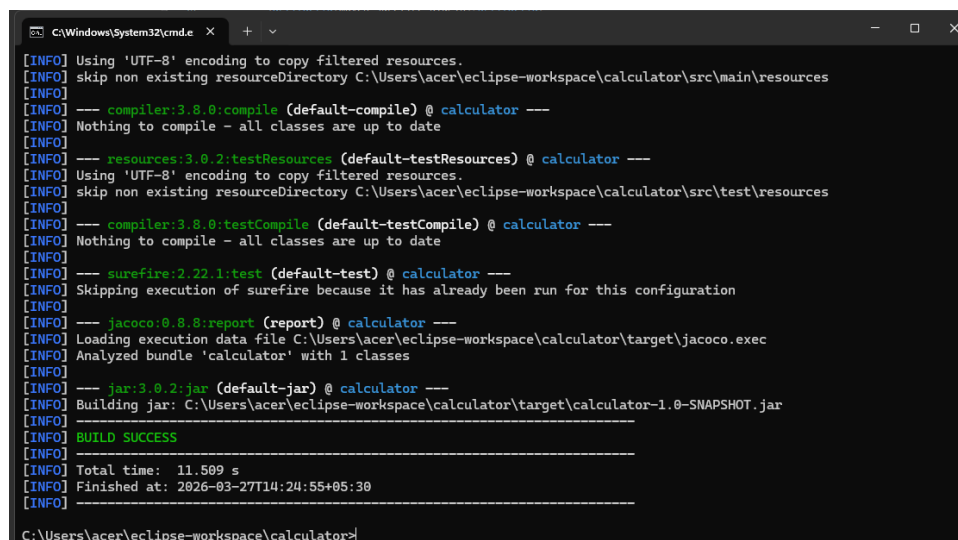


Fig 6.3 Build Success

calculator Sessions

calculator

Element	Missed Instructions	Cov	Missed Branches	Cov	Missed	Only	Missed	Lines	Missed	Methods	Missed	Classes
com.calculator	7 of 7	0%	0 of 0	n/a	2	2	3	3	2	2	1	1
Total	7 of 7	0%	0 of 0	n/a	2	2	3	3	2	2	1	1

Created with JCCoCo 0.3.0.202304050719

Fig 6.4 Reports

RESULT

The Maven build automation was successfully configured and executed, generating automated test reports, code coverage metrics, a SonarQube analysis, and the final deployable .war artifact.

EX.NO: 7

DATE:

JENKINS INSTALLATION AND SYSTEM CONFIGURATION

AIM

To configure Jenkins for Continuous Integration (CI) by setting up global tools and environment variables.

PROCEDURE

Configuring Jenkins Global Settings

- **Step 1:** Start the Jenkins server and access the dashboard by entering the URL <http://localhost:8064/jenkins> in a web browser.
- **Step 2:** Navigate to **Manage Jenkins -> Configure System** to access global settings and paths.
- **Step 3:** Observe and configure the Jenkins Maven integration.

Additional Tool Configurations

- **Step 4:** Provide the tool configurations for JDK and Maven by navigating to **Manage Jenkins -> Global Tool Configuration**.
- **Step 5:** Navigate to Global Properties (Manage Jenkins -> Configure System -> Global properties -> Environment variables) and set the JAVA_HOME and M2_HOME paths to match the respective directories on your local machine.

OUTPUT

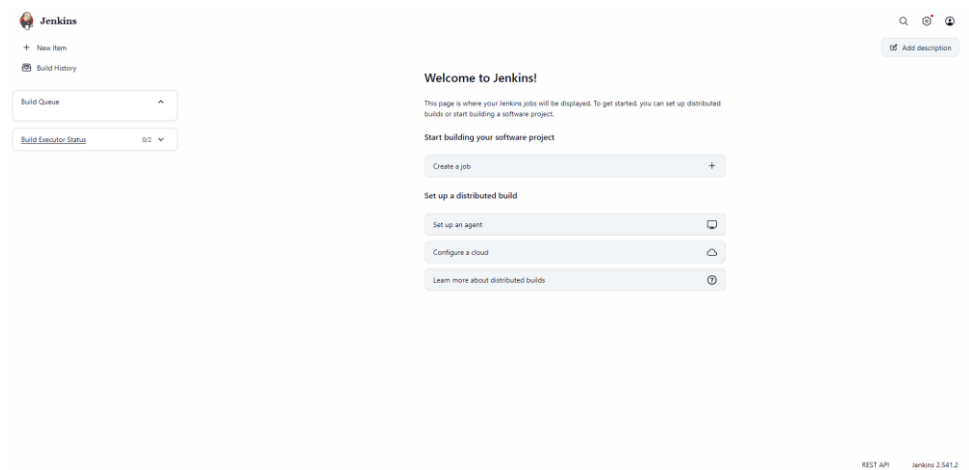


Fig 7.1 Jenkins Dashboard

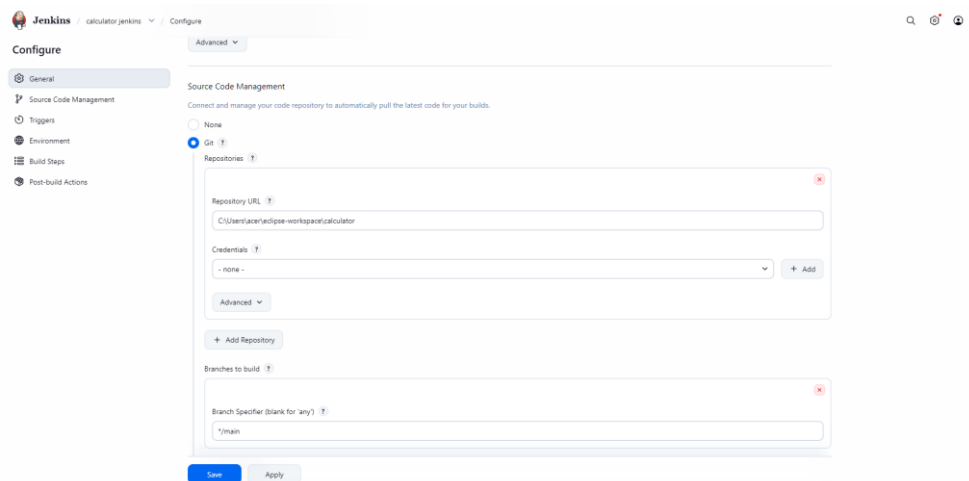


Fig 7.2 Configuring Setting

RESULT

The Jenkins server was successfully configured with the necessary system environment variables and global tool paths.

EX.NO: 8

DATE:

DOWNLOAD THE PLUGINS IN JENKINS

AIM

To download and install the required plugins in Jenkins to support the CI/CD pipeline tools.

PROCEDURE

Installing Plugins

- **Step 1:** Go to the Jenkins dashboard URL.
- **Step 2:** Navigate to **Manage Jenkins -> Manage Plugins**.
- **Step 3:** Under the **Available** tab in the plugin manager, search for the **Copy Artifact** plugin and check the corresponding box.
- **Step 4:** Repeat the search process to select the following additional plugins:
 - JaCoCo
 - Artifactory
 - Build pipeline
 - Deploy to container
- **Step 5:** Click on **Install without restart** to add the plugins to the Jenkins server.

OUTPUT

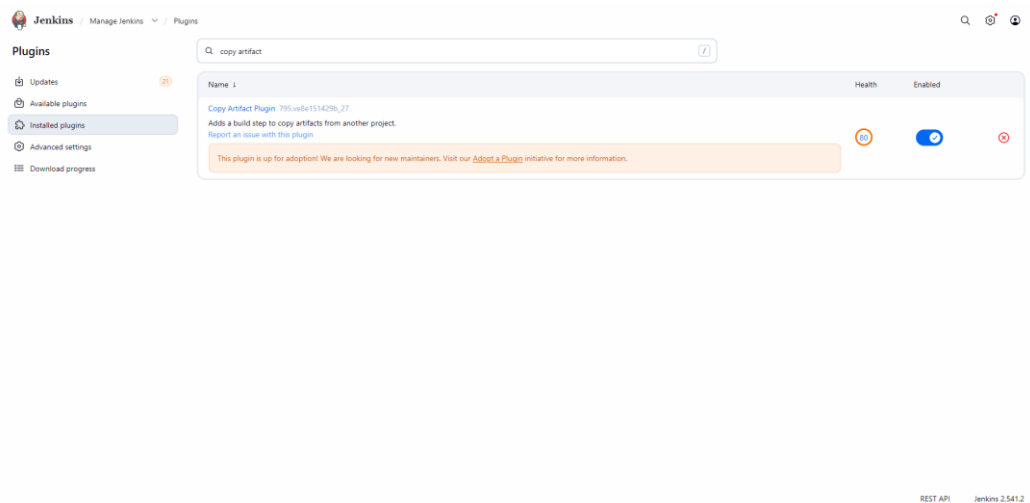


Fig 8.1 Installing copy artifact plugin

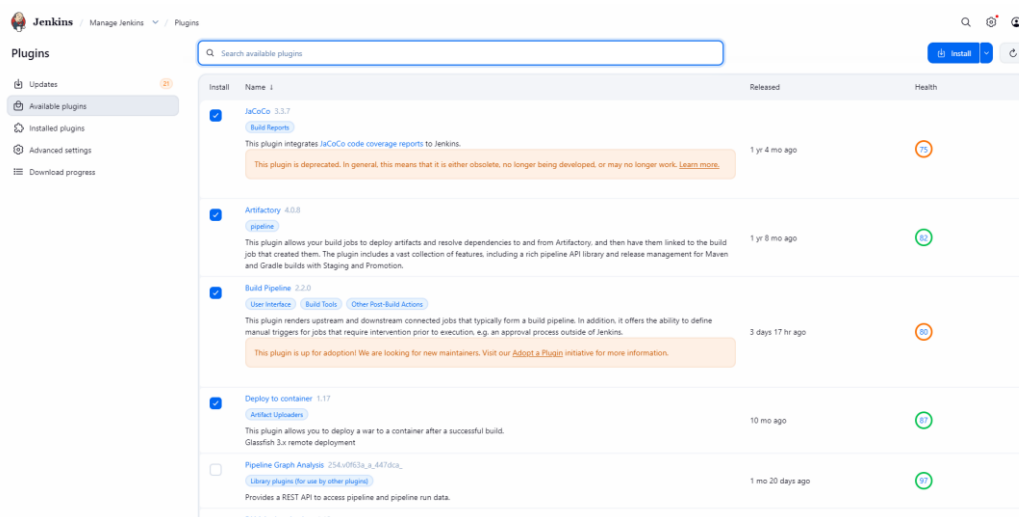


Fig 8.2 Installing other plugins

RESULT

The necessary plugins for the pipeline (Copy Artifact, JaCoCo, Artifactory, Build pipeline, and Deploy to container) were successfully downloaded and installed in Jenkins.

EX.NO: 9

DATE:

CREATING CENTRAL CI PIPELINE

AIM

To create a main line Continuous Integration (CI) pipeline by chaining multiple Jenkins jobs together.

PROCEDURE

Creating the Pipeline Folder

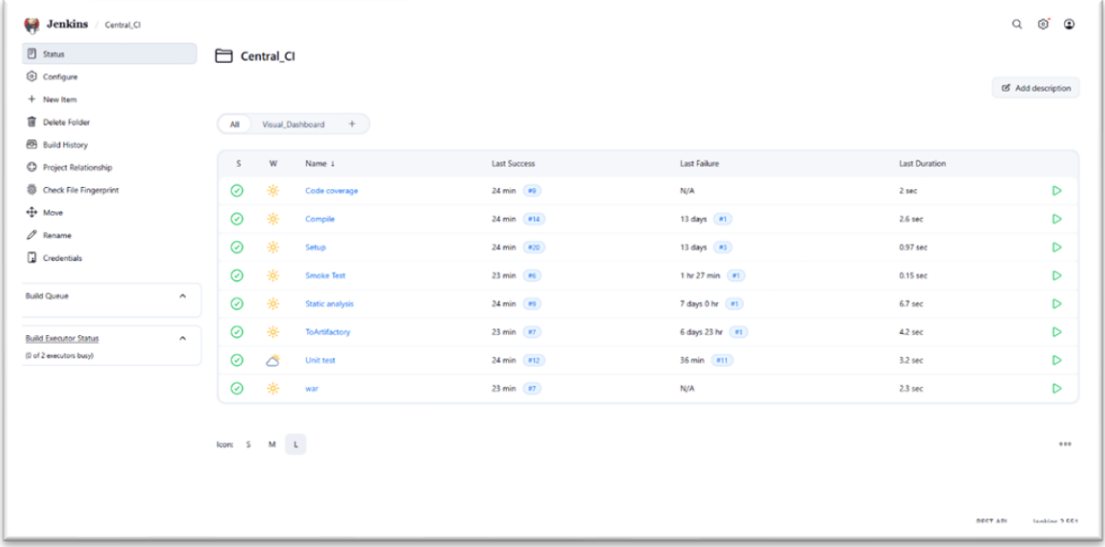
- **Step 1:** Use the "New Item" option to create a new Folder item named Central_CI to house the pipeline.
- **Step 2:** Add jobs of type "Freestyle Project" inside this folder for each task needed in the pipeline.

Configuring the Pipeline Jobs

- **Step 3 (Setup Job):** Create a job named Setup. Configure the Source Code Management to pull code from the central Git repository by entering the repository URL and credentials.
- **Step 4 (Compile Job):** Create a job named Compile. Configure it to copy artifacts from the Setup job and use the "Invoke top-level Maven targets" build step to compile the code.
- **Step 5 (Unit Test Job):** Create a job named Unit test. Configure it to run automated testing targets and archive the test artifacts.
- **Step 6 (Code Coverage Job):** Create a job named Code coverage. Configure it to invoke the jacoco:report Maven target to generate test coverage metrics.
- **Step 7 (Static Analysis Job):** Create a job named Static analysis. Configure it to invoke the sonar:sonar Maven target to push the code and coverage reports to the SonarQube server.
- **Step 8 (Package Job):** Create a job named war. Configure it to package the tested and analyzed application into a deployable .war file archive.

- **Step 9 (Artifactory Job):** Create a job named ToArtifactory. Configure the Maven build step to deploy the final package to the JFrog Artifactory local repository.
- **Step 10 (Smoke Test Job):** Create a final job named Smoke Test to verify the deployment. Link all jobs sequentially using the "Build other projects" Post-build action.

OUTPUT



The image shows the Jenkins Pipeline Jobs dashboard for a project named 'Central_CI'. The dashboard lists several pipeline jobs with their status, last success, last failure, and last duration. The jobs are: Code coverage, Compile, Setup, Smoke Test, Static analysis, ToArtifactory, Unit test, and war. All jobs are shown as successful (green checkmark) and have a duration of 24 min or less.

S	W	Name	Last Success	Last Failure	Last Duration
✓	☀	Code coverage	24 min #1	N/A	2 sec
✓	☀	Compile	24 min #14	13 days #1	2.6 sec
✓	☀	Setup	24 min #26	13 days #1	0.97 sec
✓	☀	Smoke Test	23 min #6	1 hr 27 min #1	0.15 sec
✓	☀	Static analysis	24 min #5	7 days 0 hr #1	6.7 sec
✓	☀	ToArtifactory	23 min #1	6 days 23 hr #1	4.2 sec
✓	☀	Unit test	24 min #12	36 min #11	3.2 sec
✓	☀	war	23 min #7	N/A	3.3 sec

Fig 9.1 Pipeline Jobs



Fig 9.2 Final Ouput

RESULT

The Central CI pipeline was successfully created and orchestrated, linking code checkout, compilation, unit testing, code coverage, static analysis, and artifact deployment into a continuous automated chain.

EX.NO: 10	COPYING AND MOVING JOBS
DATE:	

AIM

To understand how to duplicate existing Jenkins jobs and move them to different folder locations within the Jenkins environment.

PROCEDURE

Copying Jobs

- **Step 1:** Click on the "New Item" option from the left panel in Jenkins.
- **Step 2:** Enter a name for the new job, scroll to the "Copy from" field at the bottom, and type the name of the existing job you wish to duplicate.
- **Step 3:** Click "OK" and observe the newly copied job.

Moving Jobs

- **Step 1:** Navigate to the folder containing the jobs that need to be moved.
- **Step 2:** Click on the "Move" option from the left panel.
- **Step 3:** Select the destination folder from the dropdown menu where you want to move the jobs.
- **Step 4:** Click on "Move" and verify that the jobs are now located in the new directory.

OUTPUT

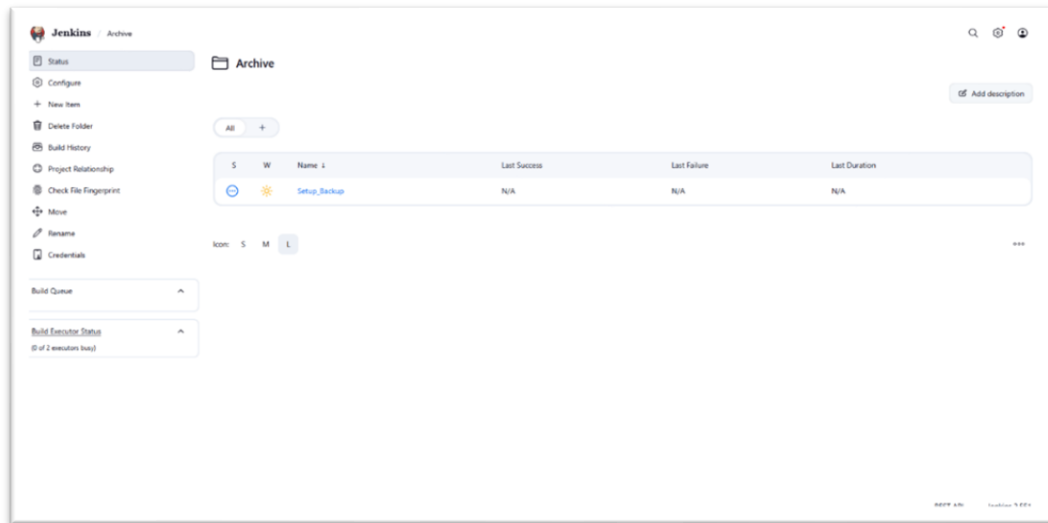


Fig 10.1 Copying Job

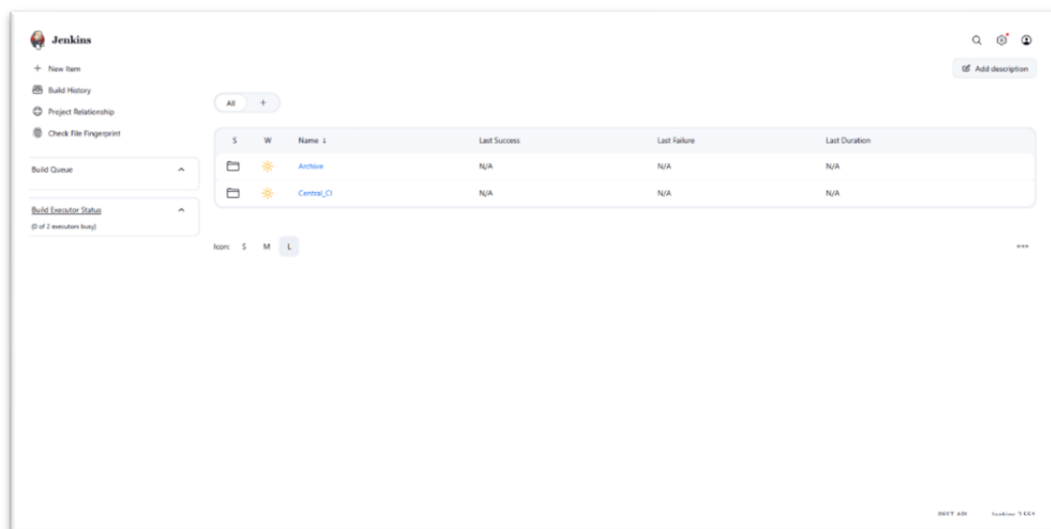


Fig 10.2 Moving Job

RESULT

Jenkins jobs were successfully copied to create duplicates and moved to new folder locations for better organization.

EX.NO: 11	CREATING PIPELINE VIEW IN JENKINS
DATE:	

AIM

To create a visual pipeline view in Jenkins to graphically monitor the flow of the Continuous Integration jobs.

PROCEDURE

Creating the Pipeline View

- **Step 1:** Click on the '+' symbol tab located inside your Jenkins project folder.
- **Step 2:** Provide a name for the pipeline in the "View name" field, select the "Build Pipeline View" option, and click "Create".
- **Step 3:** In the view configuration under "Upstream/downstream config", select your pipeline's initial starting job (e.g., the Setup job) as the "Select Initial Job" and click OK.

Executing the View

- **Step 4:** Execute the initial job and navigate to your new view to observe the visual pipeline flow as the chained jobs execute sequentially.

OUTPUT



Fig 11.1 Build Pipeline view

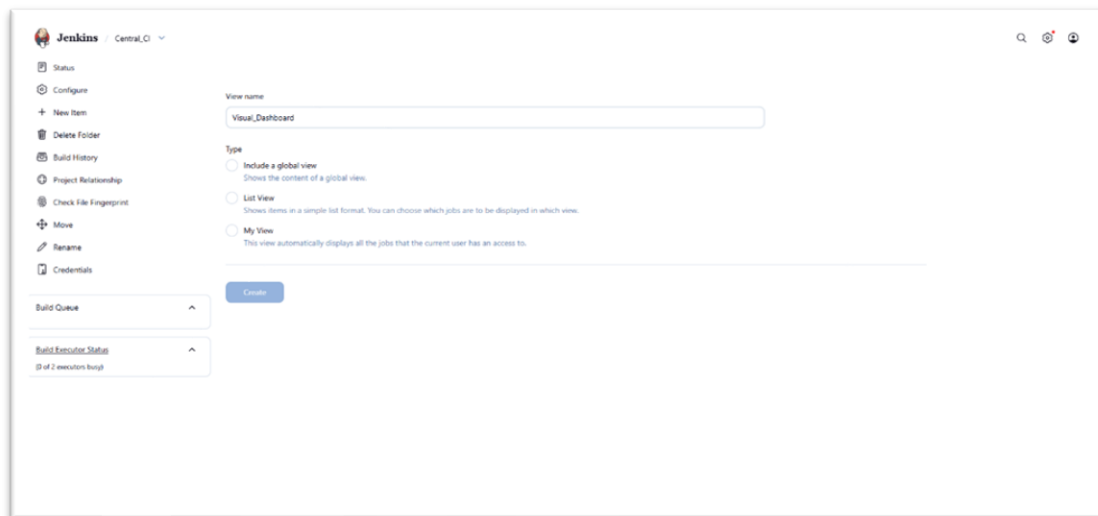


Fig 11.2 Creating Pipeline view

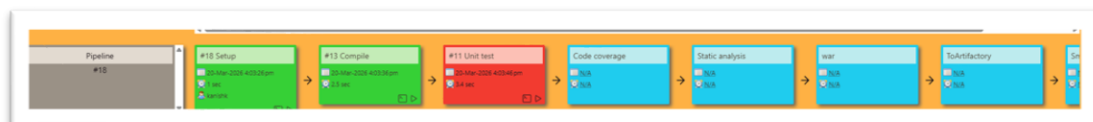


Fig 11.3 Pipeline

RESULT

A Build Pipeline view was successfully created in Jenkins, providing a graphical representation of the continuous integration process.

EX.NO: 12

DATE:

CONFIGURING GATE CONDITIONS

AIM

To configure gating conditions in Jenkins using SonarQube, unit tests, and JaCoCo coverage reports to block deployments if code quality or stability standards are not met.

PROCEDURE

Configuring Quality Gates in SonarQube

- **Step 1:** Log into the SonarQube server (<http://localhost:9000>) as an administrator.
- **Step 2:** Navigate to "Quality Gates", click "Create", and name the new gate (e.g., "CALQG").
- **Step 3:** Add specific metric conditions that will fail the gate, such as "Complexity > 100", "Critical Issues > 10", "Duplicated lines > 50", "Major issues > 10", and "Technical Debt Ratio > 1.4" .
- **Step 4:** Link the newly created Quality Gate to your specific project from the interface.
- **Step 5:** In SonarQube settings, navigate to the "build breaker" configuration to ensure that a quality gate violation actively causes the Maven target to fail.
- **Step 6:** In Jenkins, edit the Static Analysis job and set the "Build other projects" post-build action to trigger the next job *only* if the build is stable.

Configuring Gating for Unit Testing

- **Step 1:** In your Jenkins Unit Test job, configure the "Publish JUnit test result report" action. Set up the health report amplification factor to define how test failures impact the build stability.
- **Step 2:** Ensure the downstream job trigger is set to "Trigger only if build is stable".

Testing the Gates

- **Step 1:** To test the gate, deliberately modify a unit test in your source code (e.g., edit CalculatorTest.java to contain a failing assertion like `assertTrue(c1.doadd(1, 2) == 4);`).

- **Step 2:** Commit the code and trigger the Jenkins build. Observe the pipeline fail due to the broken test.
- **Step 3:** Correct the test code, commit, and observe that the build passes the gates and remains stable.

OUTPUT



Fig 12.1 Pipeline

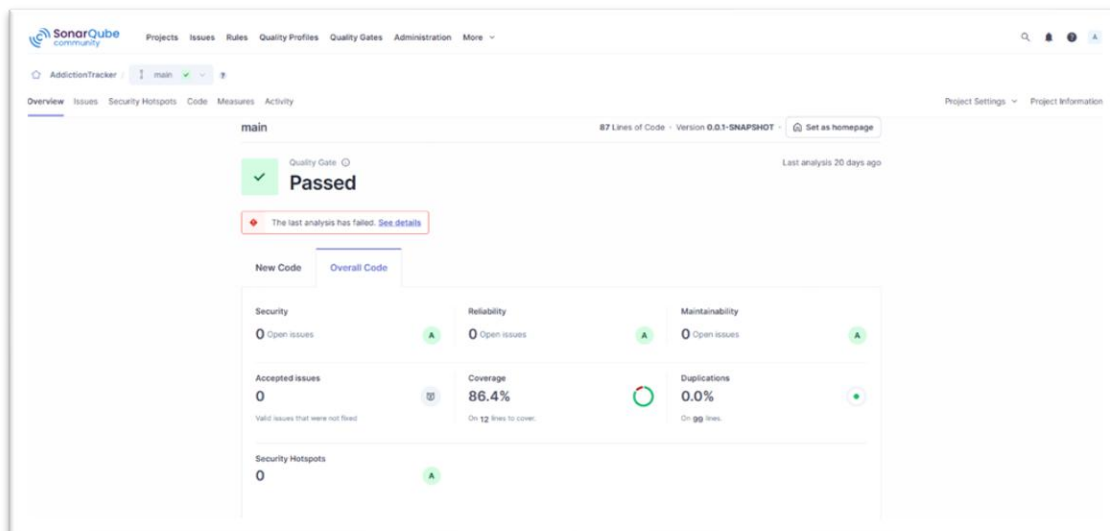


Fig 12.2 Testing

RESULT

Gating conditions were successfully applied using SonarQube and Jenkins configurations to ensure that strict code quality limits are maintained and that tests pass in every build.